VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

Aayush Kumar Shah

(1BF24CS007)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"OPERATING SYSTEMS"** carried out by **Aayush Kumar shah(1BF24CS007)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-
26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab **(23CS4PCOPS)** work prescribed for the said degree.

Sandhya A Kulkarni
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	**Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF c) Priority d) Round Robin**	
2	**Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.**	
3	**Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate-Monotonic b) Earliest-deadline First c) Proportional scheduling**	
4	**Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem**	
5	**Write a C program to simulate: a) Banker's algorithm for the purpose of deadlock avoidance. b) Deadlock Detection**	
6	**Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit**	
7	**Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal**	

Course outcomes:

CO1	Apply the different concepts and functionalities of Operating System.
CO2	Analyse various Operating system strategies and techniques.
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

CODE:

```
#include <stdio.h>

int main() {
   int n, bt[20], wt[20], tat[20], at[20], ct[20], pid[20];
   int i, j, temp;
   float twt = 0.0, ttat = 0.0, awt, att;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      { pid[i] = i + 1;
      printf("Enter Arrival Time for P%d: ", i + 1);
      scanf("%d", &at[i]);
      printf("Enter Burst Time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }
   for(i = 0; i < n - 1; i++)
      { for(j = i + 1; j < n; j++)
      {
         if(at[i] > at[j]) {
            // Swap arrival times
            temp = at[i]; at[i] = at[j]; at[j] = temp;
            // Swap burst times
            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            // Swap process IDs
            temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;
         }
      }
   }
```

```
 ct[0] = at[0] + bt[0];
   tat[0] = ct[0] - at[0];
   wt[0] = tat[0] - bt[0];

   for(i = 1; i < n; i++)
      { if(ct[i - 1] < at[i]) {
         ct[i] = at[i] + bt[i];
      } else {
         ct[i] = ct[i - 1] + bt[i];
      }
      tat[i] = ct[i] - at[i];
      wt[i] = tat[i] - bt[i];
   }
   for(i = 0; i < n; i++)
      { twt += wt[i];
      ttat += tat[i];
   }
   awt = twt / n;
   att = ttat / n;
   printf("\nP\tAT\tBT\tCT\tWT\tTAT");
   for(i = 0; i < n; i++) {
      printf("\nP%d\t%d\t%d\t%d\t%d\t%d",
            pid[i], at[i], bt[i], ct[i], wt[i], tat[i]);
   }
   printf("\n\nAverage Waiting Time: %.2f", awt);
   printf("\nAverage Turnaround Time: %.2f\n", att);

   return 0;
}
```

Output

```
Enter number of processes: 3
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 24
Enter Arrival Time for P2: 0
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 0
Enter Burst Time for P3: 3

P       AT      BT      CT      WT      TAT
P1      0       24      24      0       24
P2      0       3       27      24      27
P3      0       3       30      27      30

Average Waiting Time: 17.00
Average Turnaround Time: 27.00

Process returned 0 (0x0)   execution time : 15.553 s
Press any key to continue.
```

b) SJF (Preemptive)

Code

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], rt[10];
   int i, time = 0, completed = 0, smallest, prev = -1;
   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      { p[i] = i + 1;
      printf("\nProcess %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      bt_copy[i] = bt[i];
      rt[i] = bt[i];
   }

   printf("\nGantt Chart: ");

   while(completed < n) {
      smallest = -1;
      int min_rt = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt)
            { min_rt = rt[i];
            smallest = i;
         }
      }

      if(smallest == -1)
         { time++;
         continue;
      }
      if(prev != p[smallest])
         { printf("P%d ",
         p[smallest]); prev =
         p[smallest];
```

}

```
        rt[smallest]--;  time++;
        if(rt[smallest] == 0) {
            ct[smallest] = time;
            tat[smallest] = ct[smallest] - at[smallest];
            wt[smallest] = tat[smallest] - bt[smallest];
            completed++;
        }
    }

    printf("\n\nP\tAT\tBT\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i], at[i], bt_copy[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2

Process 2
Arrival Time: 10
Burst Time: 2

Process 3
Arrival Time: 3
Burst Time: 9

Gantt Chart: P3 P1 P2 P3

P       AT      BT      CT      WT      TAT
P1      10      2       12      0       2
P2      10      2       14      2       4
P3      3       9       16      4       13

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 30.911 s
Press any key to continue.
```

b) SJF (Non-Preemptive)

Code

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10];
   int i, j, time = 0, completed = 0, smallest;
   float avg_wt = 0, avg_tat = 0;
   printf("Enter number of processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
      { p[i] = i + 1;
      printf("Process %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      bt_copy[i] = bt[i];
   }
   while(completed < n)
      { smallest = -1;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && bt[i] > 0) {
            if(smallest == -1 || bt[i] < bt[smallest])
               { smallest = i;
            }
         }
      }
      if(smallest == -1)
         { time++;
         continue;
      }
      time += bt[smallest];
      ct[smallest] = time;
      tat[smallest] = ct[smallest] - at[smallest];
      wt[smallest] = tat[smallest] - bt[smallest];

      bt[smallest] = 0;
      completed++;
   }

   printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");
```

```
    for(i = 0; i < n; i++)
        { printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i], at[i], bt_copy[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3
Process 1
Arrival Time: 10
Burst Time: 2
Process 2
Arrival Time: 10
Burst Time: 2
Process 3
Arrival Time: 3
Burst Time: 9

P       AT      BT      CT      WT      TAT
P1      10      2       14      2       4
P2      10      2       16      4       6
P3      3       9       12      0       9

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 8.111 s
Press any key to continue.
```

c) Priority (Preemptive)

Code

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], pr[10], rt[10];
   int i, time = 0, completed = 0, highest, prev = -1;
   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      { p[i] = i + 1;
      printf("\nProcess %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      printf("Priority (lower number = higher priority): ");
      scanf("%d", &pr[i]);
      bt_copy[i] = bt[i];
      rt[i] = bt[i];
   }

   printf("\nGantt Chart: ");

   while(completed < n) {
      highest = -1;
      int max_pr = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && rt[i] > 0 && pr[i] < max_pr)
            { max_pr = pr[i];
            highest = i;
         }
      }

      if(highest == -1)
         { time++;
         continue;
      }
```

```
        if(prev != p[highest])
            { printf("P%d ", p[highest]);
            prev = p[highest];
        }
        rt[highest]--;  time++;
        if(rt[highest] == 0) {
            ct[highest] = time;
            tat[highest] = ct[highest] - at[highest];
            wt[highest] = tat[highest] - bt[highest];
            completed++;
        }
    }

    printf("\n\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i], at[i], bt_copy[i], pr[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 3

Process 2
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 1

Process 3
Arrival Time: 3
Burst Time: 9
Priority (lower number = higher priority): 2

Gantt Chart: P3 P2 P3 P1

P       AT      BT      PR      CT      WT      TAT
P1      10      2       3       16      4       6
P2      10      2       1       12      0       2
P3      3       9       2       14      2       11

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 142.278 s
Press any key to continue.
```

c) Priority (Non-Preemptive)

Code

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], pr[10];
   int i, j, time = 0, completed = 0, highest;
   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      { p[i] = i + 1;
      printf("\nProcess %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      printf("Priority (lower number = higher priority): ");
      scanf("%d", &pr[i]);
      bt_copy[i] = bt[i];
   }

   printf("\nGantt Chart: ");

   while(completed < n) {
      highest = -1;
      int max_pr = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && bt[i] > 0 && pr[i] < max_pr)
            { max_pr = pr[i];
            highest = i;
         }
      }

      if(highest == -1)
         { time++;
         continue;
      }
```

```
printf("P%d ", p[highest]);
        time += bt[highest];
        ct[highest] = time;
        tat[highest] = ct[highest] - at[highest];
        wt[highest] = tat[highest] - bt[highest];

        bt[highest] = 0;
        completed++;
    }

    printf("\n\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i], at[i], bt_copy[i], pr[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 3

Process 2
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 1

Process 3
Arrival Time: 3
Burst Time: 9
Priority (lower number = higher priority): 2

Gantt Chart: P3 P2 P1

P       AT      BT      PR      CT      WT      TAT
P1      10      2       3       16      4       6
P2      10      2       1       14      2       4
P3      3       9       2       12      0       9

Average Waiting Time: 2.00
Average Turnaround Time: 6.33
```

d) Round Robin

Code

```
#include <stdio.h>

int main() {
   int n, quantum, i, time = 0, remain, flag = 0;
   int bt[10], at[10], wt[10], tat[10], rt[10], ct[10];
   int total_wt = 0, total_tat = 0;
   int gantt[100], gantt_time[100], gc_index = 0;

   printf("Enter total number of processes: ");
   scanf("%d", &n);
   remain = n;

   for(i = 0; i < n; i++) {
      printf("Enter Arrival Time and Burst Time for process P%d: ", i+1);
      scanf("%d %d", &at[i], &bt[i]);
      rt[i] = bt[i]; wt[i]
      = 0;
   }

   printf("Enter Time Quantum: ");
   scanf("%d", &quantum);

   printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

   for(i = 0; remain != 0;) {
      if(at[i] <= time && rt[i] > 0) {
         if(rt[i] <= quantum)
            { time += rt[i];
            rt[i] = 0;
            remain--; ct[i]
            = time;
            tat[i] = ct[i] - at[i];
            wt[i] = tat[i] - bt[i];
            total_wt += wt[i];
            total_tat += tat[i];
            printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i],
wt[i]);
```

```
gantt[gc_index] = i+1;
                gantt_time[gc_index] = time;
                gc_index++;
                flag = 1;
            } else {
                rt[i] -= quantum;
                time += quantum;
                gantt[gc_index] = i+1;
                gantt_time[gc_index] = time;
                gc_index++;
                flag = 1;
            }
        }

        i++;
        if(i == n)
            { if(flag == 0)
            {
                time++;
            }
            i = 0;
            flag = 0;
        }
    }
    printf("\nGantt Chart:\n|");
    for(i = 0; i < gc_index; i++) {
        printf(" P%d |", gantt[i]);
    }
    printf("\n0");
    for(i = 0; i < gc_index; i++)
        { printf("  %d",
        gantt_time[i]);}
    printf("\n\nAverage Turnaround Time = %.2f", (float)total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", (float)total_wt/n);
    return 0;
}
```

Output

```
Enter total number of processes: 6
Enter Arrival Time and Burst Time for process P1: 5 5
Enter Arrival Time and Burst Time for process P2: 4 6
Enter Arrival Time and Burst Time for process P3: 3 7
Enter Arrival Time and Burst Time for process P4: 1 9
Enter Arrival Time and Burst Time for process P5: 2 2
Enter Arrival Time and Burst Time for process P6: 3 6
Enter Time Quantum: 4

Process AT      BT      CT      TAT     WT
P5      2       2       7       5       3
P6      3       6       29      26      20
P1      5       5       30      25      20
P2      4       6       32      28      22
P3      3       7       35      32      25
P4      1       9       36      35      26

Gantt Chart:
| P4 | P5 | P6 | P1 | P2 | P3 | P4 | P6 | P1 | P2 | P3 | P4 |
0   5   7   11   15   19   23   27   29   30   32   35   36

Average Turnaround Time = 25.17
Average Waiting Time = 19.33

Process returned 0 (0x0)   execution time : 18.206 s
Press any key to continue.
```

```
Enter total number of processes: 3
Enter Arrival Time and Burst Time for process P1: 2 1
Enter Arrival Time and Burst Time for process P2: 4 3
Enter Arrival Time and Burst Time for process P3: 6 5
Enter Time Quantum: 3

Process AT      BT      CT      TAT     WT
P1      2       1       3       1       0
P2      4       3       7       3       0
P3      6       5       12      6       1

Gantt Chart:
| P1 | P2 | P3 | P3 |
0   3   7   10   12

Average Turnaround Time = 3.33
Average Waiting Time = 0.33

Process returned 0 (0x0)   execution time : 12.655 s
Press any key to continue.
```

Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>
#define MAX 100

int main()
   { int n, i, j;
   int PID[MAX], AT[MAX], BT[MAX], TYPE[MAX];
   int CT[MAX], TAT[MAX], WT[MAX], Start[MAX];
   int systemQueue[MAX], userQueue[MAX];
   int sysCount = 0, userCount = 0;

   int time = 0, temp;
   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
      { printf("\nProcess %d\n", i + 1);
      printf("PID: ");
      scanf("%d", &PID[i]);
      printf("Arrival Time: ");
      scanf("%d", &AT[i]);
      printf("Burst Time: ");
      scanf("%d", &BT[i]);
      printf("Type (0 = System, 1 = User): ");
      scanf("%d", &TYPE[i]);
   }
```

```
for(i = 0; i < n; i++)
   { if(TYPE[i] == 0)
      systemQueue[sysCount++] = i;
   else
      userQueue[userCount++] = i;
}


for(i = 0; i < sysCount - 1; i++)
   { for(j = i + 1; j < sysCount; j++)
   {
      if(AT[systemQueue[i]] > AT[systemQueue[j]])
         { temp = systemQueue[i];
         systemQueue[i] = systemQueue[j];
         systemQueue[j] = temp;
      }
   }
}


for(i = 0; i < userCount - 1; i++)
   { for(j = i + 1; j < userCount; j++)
   {
      if(AT[userQueue[i]] > AT[userQueue[j]])
         { temp = userQueue[i];
         userQueue[i] = userQueue[j];
         userQueue[j] = temp;
      }
   }
}


for(i = 0; i < sysCount; i++)
   { int p = systemQueue[i];

   if(time < AT[p])
      time = AT[p];

   Start[p] = time;
   time += BT[p];
   CT[p] = time;

   TAT[p] = CT[p] - AT[p];
   WT[p] = TAT[p] - BT[p];
```

```
}
```

```
    for(i = 0; i < userCount; i++)
        { int p = userQueue[i];

        if(time < AT[p])
            time = AT[p];

        Start[p] = time;
        time += BT[p];
        CT[p] = time;

        TAT[p] = CT[p] - AT[p];
        WT[p] = TAT[p] - BT[p];
    }


    printf("\nPID\tAT\tBT\tTYPE\tCT\tTAT\tWT\n");

    for(i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            PID[i], AT[i], BT[i], TYPE[i],
            CT[i], TAT[i], WT[i]);
    }

    return 0;
}
```

Output

```
Enter number of processes: 5

Process 1
PID: 1
Arrival Time: 0
Burst Time: 5
Type (0 = System, 1 = User): 0

Process 2
PID: 2
Arrival Time: 1
Burst Time: 3
Type (0 = System, 1 = User): 1

Process 3
PID: 3
Arrival Time: 2
Burst Time: 2
Type (0 = System, 1 = User): 0

Process 4
PID: 4
Arrival Time: 6
Burst Time: 2
Type (0 = System, 1 = User): 0

Process 5
PID: 5
Arrival Time: 8
Burst Time: 1
Type (0 = System, 1 = User): 1

PID     AT      BT      TYPE    CT      TAT     WT
1       0       5       0       5       5       0
2       1       3       1       12      11      8
3       2       2       0       7       5       3
4       6       2       0       9       3       1
5       8       1       1       13      5       4

Process returned 0 (0x0)   execution time : 57.064 s
Press any key to continue.
```

Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:
a) Rate-Monotonic
b) Earliest-deadline First
c) Proportional scheduling

Code:

```
#include <stdio.h>
#include <stdlib.h>

void rate_monotonic(int ids[], int periods[], int exec_times[], int n, int time)
   { int remaining[10], next_release[10];
   int current_time = 0, completed_jobs = 0;

   for (int i = 0; i < n; i++)
      { remaining[i] =
      exec_times[i]; next_release[i]
      = 0;
   }

   printf("\nRate Monotonic Scheduling\n");

   while (current_time < time) {
      int selected = -1;
      int min_period = 999999;
      for (int i = 0; i < n; i++) {
         if (next_release[i] <= current_time && remaining[i] > 0)
            { if (periods[i] < min_period) {
               min_period = periods[i];
               selected = i;
            }
         }
      }
```

```
if (selected != -1) {
        printf("Time %d: Task %d\n", current_time, ids[selected]);
        remaining[selected]--;

        if (remaining[selected] == 0)
            { next_release[selected] += periods[selected];
            remaining[selected] = exec_times[selected];
            completed_jobs++;
        }
    } else {
        printf("Time %d: Idle\n", current_time);
    }

    current_time++;
  }
}

void earliest_deadline_first(int ids[], int periods[], int exec_times[], int
deadlines[], int n, int time) {
  int remaining[10], next_release[10];
  int current_time = 0, completed_jobs = 0;

  for (int i = 0; i < n; i++)
    { remaining[i] =
    exec_times[i]; next_release[i]
    = 0;
  }

  printf("\nEarliest Deadline First Scheduling\n");

  while (current_time < time) {
    int selected = -1;
    int min_deadline = 999999;

    for (int i = 0; i < n; i++) {
      if (next_release[i] <= current_time && remaining[i] > 0)
        { int absolute_deadline = next_release[i] + deadlines[i];
        if (absolute_deadline < min_deadline)
          { min_deadline = absolute_deadline;
          selected = i;
        }
      }
```

}

```
        if (selected != -1) {
            printf("Time %d: Task %d\n", current_time, ids[selected]);
            remaining[selected]--;

            if (remaining[selected] == 0)
                { next_release[selected] += periods[selected];
                remaining[selected] = exec_times[selected];
                completed_jobs++;
            }
        } else {
            printf("Time %d: Idle\n", current_time);
        }

        current_time++;
    }
}

void proportional_scheduling(int ids[], int weights[], int n, int time)
    { int total_weight = 0;
    for (int i = 0; i < n; i++)
        { total_weight += weights[i];
    }
    int slots[100] = {0};
    int *share = (int *)calloc(n, sizeof(int));

    for (int i = 0; i < n; i++) {
        share[i] = (weights[i] * time + total_weight / 2) / total_weight;
    }
    int current_time = 0;
    int done = 0;

    while (!done)
        { done = 1;
        for (int i = 0; i < n && current_time < time; i++)
            { if (share[i] > 0) {
                slots[current_time++] = ids[i];
                share[i]--;
                done = 0;
            }
        }
    }
```

```
    while (current_time < time)
        { slots[current_time++] = -1;
    }
    printf("\nProportional Scheduling\n");
    for (int i = 0; i < time; i++) {
        if (slots[i] != -1) {
            printf("Time %d: Task %d\n", i, slots[i]);
        } else {
            printf("Time %d: Idle\n", i);
        }
    }

    free(share);
}

int main() {
    int ids[10], periods[10], exec_times[10], deadlines[10], weights[10];
    int n, time;

    printf("Enter number of tasks: ");
    scanf("%d", &n);
    printf("Enter simulation time: ");
    scanf("%d", &time);

    for (int i = 0; i < n; i++) { ids[i]
        = i + 1;
        printf("\nTask %d:\n", i+1);
        printf("  Period: ");
        scanf("%d", &periods[i]);
        printf("  Execution time: ");
        scanf("%d", &exec_times[i]);
        printf("  Deadline: ");
        scanf("%d", &deadlines[i]);
        printf("  Weight: ");
        scanf("%d", &weights[i]);
    }
    rate_monotonic(ids, periods, exec_times, n, time);
    earliest_deadline_first(ids, periods, exec_times, deadlines, n, time);
    proportional_scheduling(ids, weights, n, time);

    return 0;
}
```

Output

```
Enter number of tasks: 3
Enter simulation time: 20

Task 1:
  Period: 6
  Execution time: 2
  Deadline: 6
  Weight: 2

Task 2:
  Period: 10
  Execution time: 3
  Deadline: 10
  Weight: 3

Task 3:
  Period: 15
  Execution time: 4
  Deadline: 15
  Weight: 1

Rate Monotonic Scheduling
Time 0: Task 1
Time 1: Task 1
Time 2: Task 2
Time 3: Task 2
Time 4: Task 2
Time 5: Task 3
Time 6: Task 1
Time 7: Task 1
Time 8: Task 3
Time 9: Task 3
Time 10: Task 2
Time 11: Task 2
Time 12: Task 1
Time 13: Task 1
Time 14: Task 2
Time 15: Task 3
Time 16: Task 3
Time 17: Task 3
Time 18: Task 1
Time 19: Task 1

Earliest Deadline First Scheduling
Time 0: Task 1
Time 1: Task 1
Time 2: Task 2
Time 3: Task 2
Time 4: Task 2
Time 5: Task 3
Time 6: Task 1
```

```
Earliest Deadline First Scheduling
Time 0: Task 1
Time 1: Task 1
Time 2: Task 2
Time 3: Task 2
Time 4: Task 2
Time 5: Task 3
Time 6: Task 1
Time 7: Task 1
Time 8: Task 3
Time 9: Task 3
Time 10: Task 3
Time 11: Task 2
Time 12: Task 1
Time 13: Task 1
Time 14: Task 2
Time 15: Task 2
Time 16: Task 3
Time 17: Task 3
Time 18: Task 1
Time 19: Task 1

Proportional Scheduling
Time 0: Task 1
Time 1: Task 2
Time 2: Task 3
Time 3: Task 1
Time 4: Task 2
Time 5: Task 3
Time 6: Task 1
Time 7: Task 2
Time 8: Task 3
Time 9: Task 1
Time 10: Task 2
Time 11: Task 1
Time 12: Task 2
Time 13: Task 1
Time 14: Task 2
Time 15: Task 1
Time 16: Task 2
Time 17: Task 2
Time 18: Task 2
Time 19: Task 2

Process returned 0 (0x0)   execution time : 13.968 s
Press any key to continue.
```

Program 4:

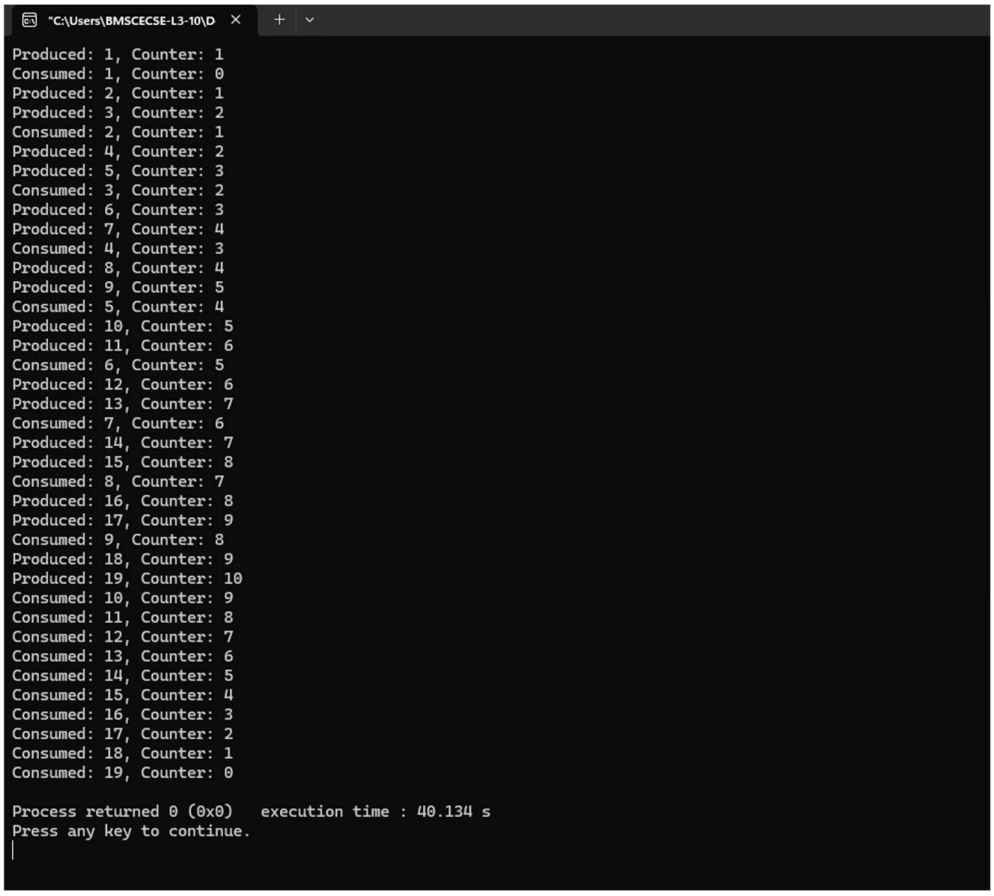
Write a C program to simulate:
a) Producer-Consumer problem using semaphores.

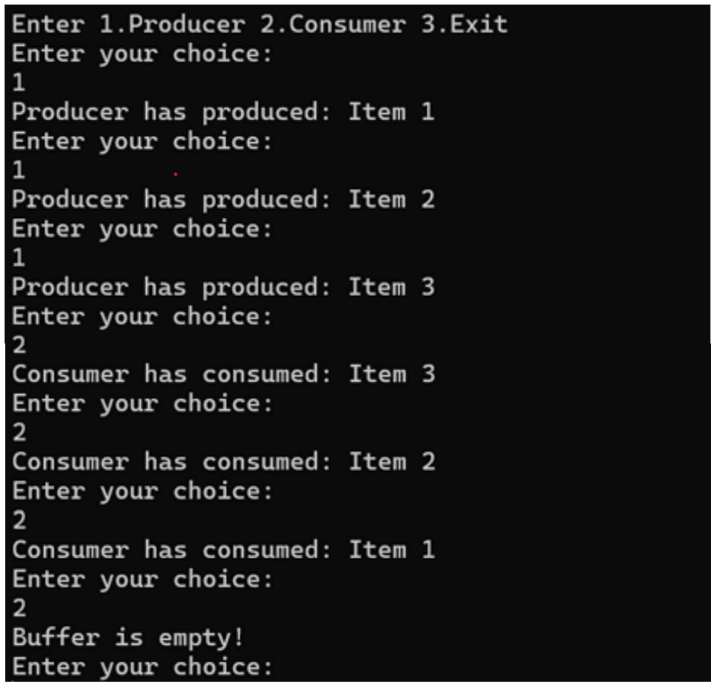
Code

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 10
#define NUM_ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int counter = 0;

sem_t empty;
sem_t full;
sem_t mutex;

void*   producer(void*   arg)
   { int item;
   for (int i = 0; i < NUM_ITEMS; i++)
      { item = i;
      sem_wait(&empty);
      sem_wait(&mutex);
      buffer[in] = item;
      in = (in + 1) % BUFFER_SIZE;
      counter++;
      printf("Produced: %d, Counter: %d\n", item, counter);
      sem_post(&mutex);
      sem_post(&full);
      sleep(1);
   }
   return NULL;
}
```

```
void* consumer(void* arg)
   { int item;
   for (int i = 0; i < NUM_ITEMS; i++)
      { sem_wait(&full);
      sem_wait(&mutex);
      item = buffer[out];
      out = (out + 1) % BUFFER_SIZE;
      counter--;
      printf("Consumed: %d, Counter: %d\n", item, counter);
      sem_post(&mutex);
      sem_post(&empty);
      sleep(2);
   }
   return NULL;
}

int main() {
   pthread_t prod, cons;
   sem_init(&empty, 0, BUFFER_SIZE);
   sem_init(&full, 0, 0);
   sem_init(&mutex, 0, 1);
   pthread_create(&prod, NULL, producer, NULL);
   pthread_create(&cons, NULL, consumer, NULL);
   pthread_join(prod, NULL);
   pthread_join(cons, NULL);
   sem_destroy(&empty);
   sem_destroy(&full);
   sem_destroy(&mutex);
   return 0;
}
```

Output


"C:\Users\BMSCECSE-L3-10\D
Produced: 1, Counter: 1
Consumed: 1, Counter: 0
Produced: 2, Counter: 1
Produced: 3, Counter: 2
Consumed: 2, Counter: 1
Produced: 4, Counter: 2
Produced: 5, Counter: 3
Consumed: 3, Counter: 2
Produced: 6, Counter: 3
Produced: 7, Counter: 4
Consumed: 4, Counter: 3
Produced: 8, Counter: 4
Produced: 9, Counter: 5
Consumed: 5, Counter: 4
Produced: 10, Counter: 5
Produced: 11, Counter: 6
Consumed: 6, Counter: 5
Produced: 12, Counter: 6
Produced: 13, Counter: 7
Consumed: 7, Counter: 6
Produced: 14, Counter: 7
Produced: 15, Counter: 8
Consumed: 8, Counter: 7
Produced: 16, Counter: 8
Produced: 17, Counter: 9
Consumed: 9, Counter: 8
Produced: 18, Counter: 9
Produced: 19, Counter: 10
Consumed: 10, Counter: 9
Consumed: 11, Counter: 8
Consumed: 12, Counter: 7
Consumed: 13, Counter: 6
Consumed: 14, Counter: 5
Consumed: 15, Counter: 4
Consumed: 16, Counter: 3
Consumed: 17, Counter: 2
Consumed: 18, Counter: 1
Consumed: 19, Counter: 0
Process returned 0 (0x0) execution time : 40.134 s
Press any key to continue.


Enter 1.Producer 2.Consumer 3.Exit
Enter your choice:
1
Producer has produced: Item 1
Enter your choice:
1
Producer has produced: Item 2
Enter your choice:
1
Producer has produced: Item 3
Enter your choice:
2
Consumer has consumed: Item 3
Enter your choice:
2
Consumer has consumed: Item 2
Enter your choice:
2
Consumer has consumed: Item 1
Enter your choice:
2
Buffer is empty!
Enter your choice:

b) Dining-Philosopher's problem

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n, hungry, pos[10], choice;

   printf("Enter the total number of philosophers: ");
   scanf("%d", &n);
   printf("How many are hungry: ");
   scanf("%d", &hungry);

   for (int i = 0; i < hungry; i++) {
      printf("Enter philosopher %d position (1 to %d): ", i + 1, n);
      scanf("%d", &pos[i]);
   }

   while (1) {
      printf("\n1. One can eat at a time\n2. Two can eat at a time\n3. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      if (choice == 1) {
         printf("Allow one philosopher to eat at any time\n");
         for (int i = 0; i < hungry; i++) {
            for (int j = 0; j < hungry; j++)
               { printf("P %d is waiting\n",
               pos[j]);
            }
            printf("P %d is granted to eat\n", pos[i]);
            printf("P %d has finished eating\n", pos[i]);
         }
      } else if (choice == 2) {
         printf("Allow two philosophers to eat at any time\n");
         for (int i = 0; i < hungry; i++) {
            printf("P %d is waiting\n", pos[i]);
         }
```

```
for (int i = 0; i < hungry; i += 2)
            { if (i + 1 < hungry) {
                printf("P %d and P %d are granted to eat\n", pos[i], pos[i+1]);
                printf("P %d and P %d have finished eating\n", pos[i], pos[i+1]);
            } else {
                printf("P %d is granted to eat\n", pos[i]);
                printf("P %d has finished eating\n", pos[i]);
            }
        }
    } else if (choice == 3)
        { exit(0);
    }
  }

  return 0;
}
```

Output

```
Enter the total number of philosophers: 5
How many are hungry: 2
Enter philosopher 1 position (1 to 5): 2 3 1 4 5
Enter philosopher 2 position (1 to 5):
1. One can eat at a time
2. Two can eat at a time
3. Exit
Enter your choice: Allow one philosopher to eat at any time
P 2 is waiting
P 3 is waiting
P 2 is granted to eat
P 2 has finished eating
P 2 is waiting
P 3 is waiting
P 3 is granted to eat
P 3 has finished eating

1. One can eat at a time
2. Two can eat at a time
3. Exit
```

Program 5

Write a C program to simulate:
a) Bankers' algorithm for the purpose of deadlock avoidance.

Code

```
#include <stdio.h>
#include <stdbool.h>

int main()
   { int n, m;

   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resource types: ");
   scanf("%d", &m);

   int available[m], max[n][m], allocation[n][m], need[n][m];
   int i, j;

   printf("Enter available resources: ");
   for (i = 0; i < m; i++) scanf("%d", &available[i]);

   printf("Enter Allocation matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < m; j++)
         { scanf("%d", &allocation[i][j]);
      }
   }

   printf("Enter Max matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < m; j++)
         { scanf("%d", &max[i][j]);
         need[i][j] = max[i][j] - allocation[i][j];
      }
   }
```

```
int pid;
printf("\nEnter process ID making a request (e.g., 1): ");
scanf("%d", &pid);

int request[m];
printf("Enter Request for resources: ");
for (j = 0; j < m; j++) scanf("%d", &request[j]);
bool possible = true;
for (j = 0; j < m; j++) {
   if (request[j] > need[pid][j] || request[j] > available[j])
      { possible = false;
      break;
   }
}

if (possible) {
   for (j = 0; j < m; j++) { available[j]
      -= request[j]; allocation[pid][j]
      += request[j]; need[pid][j] -=
      request[j];
   }
} else {
   printf("Request cannot be granted immediately.\n");
}

bool finish[n];
int work[m], safeSeq[n], count = 0;

for (i = 0; i < n; i++) finish[i] = false;
for (i = 0; i < m; i++) work[i] = available[i];

while (count < n)
   { bool found =
   false;
   for (i = 0; i < n; i++)
      { if (!finish[i]) {
         bool canAllocate = true;
         for (j = 0; j < m; j++) {
            if (need[i][j] > work[j])
               { canAllocate = false;
               break;
            }
```

}

```
            if (canAllocate) {
                for (j = 0; j < m; j++) work[j] += allocation[i][j];
                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
                printf("P%d is visited(%d%d%d)\n", i, work[0], work[1], work[2]);
// Matches [cite: 5-9]
            }
        }
    }
    if (!found) {
        printf("\nSYSTEM IS NOT IN A SAFE STATE\n");
        return 0;
    }
  }

  printf("\nSYSTEM IS IN SAFE STATE [cite: 10]\n");
  printf("The Safe Sequence is (");
  for (i = 0; i < n; i++) printf("P%d%s", safeSeq[i], (i == n - 1) ? "" : " ");
  printf(") \n");

  return 0;
}
```

Output

```
Enter number of processes: 5
Enter number of resource types: 3
Enter available resources: 3 3 2
Enter Allocation matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Max matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter process ID making a request (e.g., 1): 1
Enter Request for resources: 1 0 2
P1 is visited(532)
P3 is visited(743)
P4 is visited(745)
P0 is visited(755)
P2 is visited(1057)

SYSTEM IS IN SAFE STATE [cite: 10]
The Safe Sequence is (P1 P3 P4 P0 P2)

Process returned 0 (0x0)   execution time : 68.446 s
Press any key to continue.
```

b) Deadlock Detection

Code

```
#include <stdio.h>
#include <stdbool.h>

int main()
   { int n, m;

   printf("Enter number of processes: ");
   scanf("%d", &n);
   printf("Enter number of resource types: ");
   scanf("%d", &m);

   int available[m], allocation[n][m], request[n][m];
   int i, j;

   printf("Enter available resources: ");
   for (i = 0; i < m; i++) scanf("%d", &available[i]);

   printf("Enter Allocation matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < m; j++)
         { scanf("%d", &allocation[i][j]);
      }
   }

   printf("Enter Request matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < m; j++)
         { scanf("%d", &request[i][j]);
      }
   }

   bool finish[n];
   int work[m], safeSeq[n], count = 0;

   for (i = 0; i < n; i++)
      { finish[i] = false;
   }
```

```
for (i = 0; i < m; i++)
    { work[i] = available[i];
}

while (count < n)
    { bool found =
    false;
    for (i = 0; i < n; i++)
        { if (!finish[i]) {
            bool canAllocate = true;
            for (j = 0; j < m; j++) {
                if (request[i][j] > work[j])
                    { canAllocate = false;
                    break;
                }
            }
            if (canAllocate) {
                for (j = 0; j < m; j++) work[j] += allocation[i][j];
                safeSeq[count++] = i;
                finish[i] = true;
                found = true;
                printf("P%d is visited(%d%d%d)\n", i, work[0], work[1], work[2]);
            }
        }
    }
    if (!found) {
        printf("\nSYSTEM IS IN DEADLOCK\n");
        printf("Processes causing deadlock: ");
        for (int k = 0; k < n; k++) {
            if (!finish[k]) printf("P%d ", k);
        }
        printf("\n");
        return 0;
    }
}

printf("\nSYSTEM IS NOT IN DEADLOCK\n");
printf("The Safe Sequence is (");
for (i = 0; i < n; i++) printf("P%d%s", safeSeq[i], (i == n - 1) ? "" : " ");
printf(")\n");

return 0;
```

}

Output

```
Enter number of processes: 5
Enter number of resource types: 3
Enter available resources: 0 0 0
Enter Allocation matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2
Enter Request matrix:
0 0 0
2 0 2
0 0 1
1 0 0
0 0 2
P0 is visited(010)

SYSTEM IS IN DEADLOCK
Processes causing deadlock: P1 P2 P3 P4

Process returned 0 (0x0)   execution time : 41.778 s
Press any key to continue.
```

Program 6

Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

Code

```
#include <stdio.h>

void simulate(int b[], int n, int p[], int m, int type)
   { int alloc[20], occupied[20] = {0};
   for (int i = 0; i < m; i++) alloc[i] = -1;

   for (int i = 0; i < m; i++)
      { int idx = -1;
      for (int j = 0; j < n; j++) {
         if (!occupied[j] && b[j] >= p[i])
            { if (type == 1) {
               idx = j;
               break;
            } else if (type == 2) {
               if (idx == -1 || b[j] < b[idx]) idx = j;
            } else if (type == 3) {
               if (idx == -1 || b[j] > b[idx]) idx = j;
            }
         }
      }
      if (idx != -1) { alloc[i]
         = idx;
         occupied[idx] = 1;
      }
   }

   printf("\nProcess No.\tProcess Size\tBlock No.\n");
   for (int i = 0; i < m; i++) {
      printf("%d\t\t%d\t\t", i + 1, p[i]);
      if (alloc[i] != -1) printf("%d\n", alloc[i] + 1);
      else printf("Not Allocated\n");
```

```
    }
}

int main() {
    int b[20], p[20], b_bak[20], n, m;

    printf("Enter number of blocks: ");
    scanf("%d", &n);
    printf("Enter size of each block:\n");
    for (int i = 0; i < n; i++) scanf("%d", &b[i]);

    printf("Enter number of processes: ");
    scanf("%d", &m);
    printf("Enter size of each process:\n");
    for (int i = 0; i < m; i++) scanf("%d", &p[i]);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- FIRST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 1);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- BEST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 2);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- WORST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 3);

    return 0;
}
```

Output

```
Enter number of blocks: 5
Enter size of each block:
100
500
200
300
600
Enter number of processes: 4
Enter size of each process:
212
417
112
426

--- FIRST FIT ALLOCATION ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- BEST FIT ALLOCATION ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- WORST FIT ALLOCATION ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated

Process returned 0 (0x0)   execution time : 33.166 s
Press any key to continue.
```

Program 7

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

Code

```
#include <stdio.h>

void run(int ref[], int n, int nf, int mode) {
   int frames[10], matrix[10][50], tracker[10] = {0}, fifo_idx = 0, faults = 0;
   for (int i = 0; i < nf; i++) frames[i] = -1;

   for (int i = 0; i < n; i++)
      { int found = -1;
      for (int j = 0; j < nf; j++) if (frames[j] == ref[i]) found = j;

      if (found != -1) {
         if (mode == 2) tracker[found] = i;
      } else {
         faults++;
         int slot = -1;
         for (int j = 0; j < nf; j++) if (frames[j] == -1) { slot = j; break; }

         if (slot == -1) {
            if (mode == 1) { slot = fifo_idx; fifo_idx = (fifo_idx + 1) % nf; }
            else {
               int opt_idx[10];
               for (int j = 0; j < nf; j++)
                  { opt_idx[j] = 1e9;
                  for (int k = i + 1; k < n; k++) if (frames[j] == ref[k]) { opt_idx[j]
= k; break; }
               }
               int target = 0;
               for (int j = 1; j < nf; j++) {
                  if (mode == 2 && tracker[j] < tracker[target]) target = j;
                  if (mode == 3 && opt_idx[j] > opt_idx[target]) target = j;
               }
```

```
                slot = target;
            }
        }
        frames[slot] = ref[i];
        if (mode == 2) tracker[slot] = i;
      }
      for (int j = 0; j < nf; j++) matrix[j][i] = frames[j];
   }

   for (int i = 0; i < nf; i++)
      { for (int j = 0; j < n; j++)
         if (matrix[i][j] != -1) printf("%d\t", matrix[i][j]); else printf(" \t");
      printf("\n");
   }
   printf("Total Page Faults: %d\n", faults);
}

int main() {
   int n, nf, ref[50];
   printf("Enter number of pages: "); scanf("%d", &n);
   printf("Enter reference string:\n");
   for (int i = 0; i < n; i++) scanf("%d", &ref[i]);
   printf("Enter number of frames: "); scanf("%d", &nf);

   printf("\n--- FIFO PAGE REPLACEMENT ---\n"); run(ref, n, nf, 1);
   printf("\n--- LRU PAGE REPLACEMENT ---\n");  run(ref, n, nf, 2);
   printf("\n--- OPTIMAL PAGE REPLACEMENT ---\n"); run(ref, n, nf, 3);
   return 0;
}
```

Output

```
Enter number of pages: 6
Enter reference string:
1 4 2 3 2 1
Enter number of frames: 3

--- FIFO PAGE REPLACEMENT ---
1       1       1       3       3       3
        4       4       4       4       1
                2       2       2       2
Total Page Faults: 5

--- LRU PAGE REPLACEMENT ---
1       1       1       3       3       3
        4       4       4       4       1
                2       2       2       2
Total Page Faults: 5

--- OPTIMAL PAGE REPLACEMENT ---
1       1       1       1       1       1
        4       4       3       3       3
                2       2       2       2
Total Page Faults: 4

Process returned 0 (0x0)   execution time : 7.615 s
Press any key to continue.
```